st125333_Assignment05

October 25, 2025

[26]:
```
import torch
import torchvision
from torchvision import datasets, models, transforms
import torch.nn as nn
import torch.optim as optim
import time
import os
from copy import copy
from copy import deepcopy
import torch.nn.functional as F
import zipfile
import os
import requests
from pathlib import Path
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
```

[2]:
```
class BasicBlock(nn.Module):
    '''
    BasicBlock: Simple residual block with two conv layers
    '''
    EXPANSION = 1
    def __init__(self, in_planes, out_planes, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_planes, out_planes, kernel_size=3,␣
↪stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_planes)
        self.conv2 = nn.Conv2d(out_planes, out_planes, kernel_size=3, stride=1,␣
↪padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_planes)
        self.shortcut = nn.Sequential()
        # If output size is not equal to input size, reshape it with 1x1␣
↪convolution
        if stride != 1 or in_planes != out_planes:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, out_planes, kernel_size=1, stride=stride,␣
↪bias=False),
```

```
                nn.BatchNorm2d(out_planes)
            )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += self.shortcut(x)
        out = F.relu(out)
        return out
```

[3]:
```
class BottleneckBlock(nn.Module):
    '''
    BottleneckBlock: More powerful residual block with three convs, used for␣
 ↪Resnet50 and up
    '''
    EXPANSION = 4
    def __init__(self, in_planes, planes, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_planes, planes, kernel_size=1, bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        self.conv2 = nn.Conv2d(planes, planes, kernel_size=3, stride=stride,␣
 ↪padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(planes)
        self.conv3 = nn.Conv2d(planes, self.EXPANSION * planes, kernel_size=1,␣
 ↪bias=False)
        self.bn3 = nn.BatchNorm2d(self.EXPANSION * planes)

        self.shortcut = nn.Sequential()
        # If the output size is not equal to input size, reshape it with 1x1␣
 ↪convolution
        if stride != 1 or in_planes != self.EXPANSION * planes:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, self.EXPANSION * planes,
                          kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(self.EXPANSION * planes)
            )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = F.relu(self.bn2(self.conv2(out)))
        out = self.bn3(self.conv3(out))
        out += self.shortcut(x)
        out = F.relu(out)
        return out
```

[4]:
```
class ResNet(nn.Module):
    def __init__(self, block, num_blocks, num_classes=10):
        super().__init__()
        self.in_planes = 64
        # Initial convolution
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,␣
 ↪bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        # Residual blocks
        self.layer1 = self._make_layer(block, 64, num_blocks[0], stride=1)
        self.layer2 = self._make_layer(block, 128, num_blocks[1], stride=2)
        self.layer3 = self._make_layer(block, 256, num_blocks[2], stride=2)
        self.layer4 = self._make_layer(block, 512, num_blocks[3], stride=2)
        # FC layer = 1 layer
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.linear = nn.Linear(512 * block.EXPANSION, num_classes)

    def _make_layer(self, block, planes, num_blocks, stride):
        strides = [stride] + [1] * (num_blocks-1)
        layers = []
        for stride in strides:
            layers.append(block(self.in_planes, planes, stride))
            self.in_planes = planes * block.EXPANSION
        return nn.Sequential(*layers)

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.layer1(out)
        out = self.layer2(out)
        out = self.layer3(out)
        out = self.layer4(out)
        out = self.avgpool(out)
        out = out.view(out.size(0), -1)
        out = self.linear(out)
        return out
```

[5]:
```
def ResNet18(num_classes = 10):
    '''
    First conv layer: 1
    4 residual blocks with two sets of two convolutions each: 2*2 + 2*2 + 2*2 +␣
 ↪2*2 = 16 conv layers
    last FC layer: 1
    Total layers: 1+16+1 = 18
    '''
    return ResNet(BasicBlock, [2, 2, 2, 2], num_classes)
```

```
def ResNet34(num_classes):
    '''
    First conv layer: 1
    4 residual blocks with [3, 4, 6, 3] sets of two convolutions each: 3*2 +␣
 ↪4*2 + 6*2 + 3*2 = 32
    last FC layer: 1
    Total layers: 1+32+1 = 34
    '''
    return ResNet(BasicBlock, [3, 4, 6, 3], num_classes)


def ResNet50(num_classes = 10):
    '''
    First conv layer: 1
    4 residual blocks with [3, 4, 6, 3] sets of three convolutions each: 3*3 +␣
 ↪4*3 + 6*3 + 3*3 = 48
    last FC layer: 1
    Total layers: 1+48+1 = 50
    '''
    return ResNet(Bottleneck, [3, 4, 6, 3], num_classes)


def ResNet101(num_classes = 10):
    '''
    First conv layer: 1
    4 residual blocks with [3, 4, 23, 3] sets of three convolutions each: 3*3 +␣
 ↪4*3 + 23*3 + 3*3 = 99
    last FC layer: 1
    Total layers: 1+99+1 = 101
    '''
    return ResNet(Bottleneck, [3, 4, 23, 3], num_classes)


def ResNet152(num_classes = 10):
    '''
    First conv layer: 1
    4 residual blocks with [3, 8, 36, 3] sets of three convolutions each: 3*3 +␣
 ↪8*3 + 36*3 + 3*3 = 150
    last FC layer: 1
    Total layers: 1+150+1 = 152
    '''
    return ResNet(Bottleneck, [3, 8, 36, 3], num_classes)
```

[6]:
```
# device = torch.device("cuda:2" if torch.cuda.is_available() else "cpu")
device_index = 2
if torch.cuda.device_count() > device_index:
    device = torch.device(f"cuda:{device_index}")
```

```
else:
    device = torch.device("cpu")
```

[7]:
```
class SELayer(nn.Module):
    def __init__(self, channel, reduction=16):
        super().__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Sequential(
            nn.Linear(channel, channel // reduction, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(channel // reduction, channel, bias=False),
            nn.Sigmoid()
        )

    def forward(self, x):
        b, c, _, _ = x.size()
        y = self.avg_pool(x).view(b, c)
        y = self.fc(y).view(b, c, 1, 1)
        return x * y.expand_as(x)
```

[8]:
```
class ResidualSEBasicBlock(nn.Module):
    '''
    ResidualSEBasicBlock: Standard two-convolution residual block with an SE␣
 ↪Module between the
                          second convolution and the identity addition
    '''
    EXPANSION = 1

    def __init__(self, in_planes, out_planes, stride=1, reduction=16):
        super().__init__()
        self.conv1 = nn.Conv2d(in_planes, out_planes, kernel_size=3,␣
 ↪stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_planes)
        self.conv2 = nn.Conv2d(out_planes, out_planes, kernel_size=3,
                               stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_planes)
        self.se = SELayer(out_planes, reduction)

        self.shortcut = nn.Sequential()
        # If output size is not equal to input size, reshape it with a 1x1 conv
        if stride != 1 or in_planes != out_planes:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, out_planes,
                          kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(self.EXPANSION * out_planes)
            )
```

```
    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = self.se(out)              # se net add here
        out += self.shortcut(x)         # shortcut just plus it!!!
        out = F.relu(out)
        return out


def ResSENet18_SE(num_classes = 10):
    return ResNet(ResidualSEBasicBlock, [2, 2, 2, 2], num_classes)
```

[9]:
```
def to_float_list(seq):
    result = []
    for item in seq:
        if hasattr(item, "detach"):  # Case 1: Tensor
            result.append(item.detach().cpu().numpy().item()
                          if item.numel() == 1
                          else float(item.mean().cpu().numpy()))
        elif callable(item):         # Case 2: Method or function
            result.append(np.nan)
        else:                        # Case 3: Already float/int
            result.append(float(item))
    return result
```

[10]:
```
output_dir = "output_models"
os.makedirs(output_dir, exist_ok=True)


def train_model(model, dataloaders, criterion, optimizer, num_epochs=25,␣
 ↪weights_name='weight_save', is_inception=False):
    '''
    train_model: train a model on a dataset

            Parameters:
                    model: Pytorch model
                    dataloaders: dataset
                    criterion: loss function
                    optimizer: update weights function
                    num_epochs: number of epochs
                    weights_name: file name to save weights
                    is_inception: The model is inception net (Google LeNet) or␣
 ↪not

            Returns:
                    model: Best model from evaluation result
                    train_losses: training loss history
```

```
                val_losses: validation loss history
                train_accs: training accuracy history
                val_accs: validation accuracy history
    '''
    since = time.time()

    train_losses, val_losses = [], []
    train_accs, val_accs = [], []

    best_model_wts = deepcopy(model.state_dict())
    best_acc = 0.0

    for epoch in range(num_epochs):
        epoch_start = time.time()

        print('Epoch {}/{}'.format(epoch, num_epochs - 1))
        print('-' * 10)

        # Each epoch has a training and validation phase
        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()  # Set model to training mode
            else:
                model.eval()   # Set model to evaluate mode

            running_loss = 0.0
            running_corrects = 0

            # Iterate over data.
            for batch in dataloaders[phase]:
                if isinstance(batch, dict):
                    inputs = batch['image']
                    labels = batch['label']
                else:
                    inputs, labels = batch


                inputs = inputs.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    if is_inception and phase == 'train':
                        outputs, aux_outputs = model(inputs)
                        loss1 = criterion(outputs, labels)
                        loss2 = criterion(aux_outputs, labels)
```

```
                        loss = loss1 + 0.4 * loss2
                    else:
                        outputs = model(inputs)
                        loss = criterion(outputs, labels)

                    _, preds = torch.max(outputs, 1)

                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

            epoch_loss = running_loss / len(dataloaders[phase].dataset)
            epoch_acc = running_corrects.double() / len(dataloaders[phase].
↪dataset)
            epoch_end = time.time()

            elapsed_epoch = epoch_end - epoch_start

            print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss,␣
↪epoch_acc))
            print("Epoch time taken: ", elapsed_epoch)

            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = deepcopy(model.state_dict())
                torch.save(model.state_dict(), os.path.join(output_dir,␣
↪weights_name + ".pth"))

            if phase == 'train':
                train_losses.append(epoch_loss)
                train_accs.append(epoch_acc)
            else:
                val_losses.append(epoch_loss)
                val_accs.append(epoch_acc)

        print()

    time_elapsed = time.time() - since
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60,␣
↪time_elapsed % 60))
    print('Best val Acc: {:4f}'.format(best_acc))

    model.load_state_dict(best_model_wts)
    return model, train_losses, val_losses, train_accs, val_accs
```

[11]:
```
def plot_curves(train_acc, val_acc, train_loss, val_loss, title):
    import matplotlib.pyplot as plt
    plt.figure(figsize=(5,4))
    plt.plot(train_loss, label='Train Loss')
    plt.plot(val_loss, label='Val Loss')
    plt.legend(); plt.title(f'{title} Loss'); plt.xlabel('Epoch'); plt.
 ↪ylabel('Loss'); plt.grid(True, alpha=0.3); plt.show()
    plt.figure(figsize=(5,4))
    plt.plot(train_acc, label='Train Acc')
    plt.plot(val_acc, label='Val Acc')
    plt.legend(); plt.title(f'{title} Accuracy'); plt.xlabel('Epoch'); plt.
 ↪ylabel('Accuracy'); plt.grid(True, alpha=0.3); plt.show()
```

0.1 CIFAR 10 Dataset

[34]:
```
train_preprocess = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.49139968, 0.48215827, 0.44653124), (0.24703233, 0.
 ↪24348505, 0.26158768))])

eval_preprocess = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.49139968, 0.48215827, 0.44653124), (0.24703233, 0.
 ↪24348505, 0.26158768))])

# Download CIFAR-10 and split into training, validation, and test sets.
# The copy of the training dataset after the split allows us to keep
# the same training/validation split of the original training set but
# apply different transforms to the training set and validation set.

full_train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                                  download=True)

train_dataset, val_dataset = torch.utils.data.random_split(full_train_dataset,␣
 ↪[40000, 10000])
train_dataset.dataset = copy(full_train_dataset)
train_dataset.dataset.transform = train_preprocess
val_dataset.dataset.transform = eval_preprocess

test_dataset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                            download=True,␣
 ↪transform=eval_preprocess)

# DataLoaders for the three datasets

BATCH_SIZE=128
```

```
NUM_WORKERS=4

train_dataloader = torch.utils.data.DataLoader(train_dataset,␣
 ↪batch_size=BATCH_SIZE,
                                               shuffle=True,␣
 ↪num_workers=NUM_WORKERS)
val_dataloader = torch.utils.data.DataLoader(val_dataset, batch_size=BATCH_SIZE,
                                               shuffle=False,␣
 ↪num_workers=NUM_WORKERS)
test_dataloader = torch.utils.data.DataLoader(test_dataset,␣
 ↪batch_size=BATCH_SIZE,
                                               shuffle=False,␣
 ↪num_workers=NUM_WORKERS)

dataloaders = {'train': train_dataloader, 'val': val_dataloader}
```

[36]:
```
resnet = ResNet18().to(device)
# Optimizer and loss function
criterion = nn.CrossEntropyLoss()
params_to_update = resnet.parameters()
# Now we'll use Adam optimization
optimizer = optim.Adam(params_to_update, lr=0.01)

best_model, train_losses, val_losses, train_accs, val_accs =␣
 ↪train_model(resnet, dataloaders, criterion, optimizer, 25,␣
 ↪'resnet18_bestsofar')
```

```
Epoch 0/24
----------
train Loss: 1.8231 Acc: 0.3368
Epoch time taken:  13.97463607788086
val Loss: 1.5754 Acc: 0.4226
Epoch time taken:  15.288867235183716

Epoch 1/24
----------
train Loss: 1.3002 Acc: 0.5267
Epoch time taken:  14.046583890914917
val Loss: 1.2467 Acc: 0.5501
Epoch time taken:  15.403651475906372

Epoch 2/24
----------
train Loss: 1.0196 Acc: 0.6341
Epoch time taken:  14.18627405166626
val Loss: 0.8900 Acc: 0.6788
Epoch time taken:  15.564897060394287
```

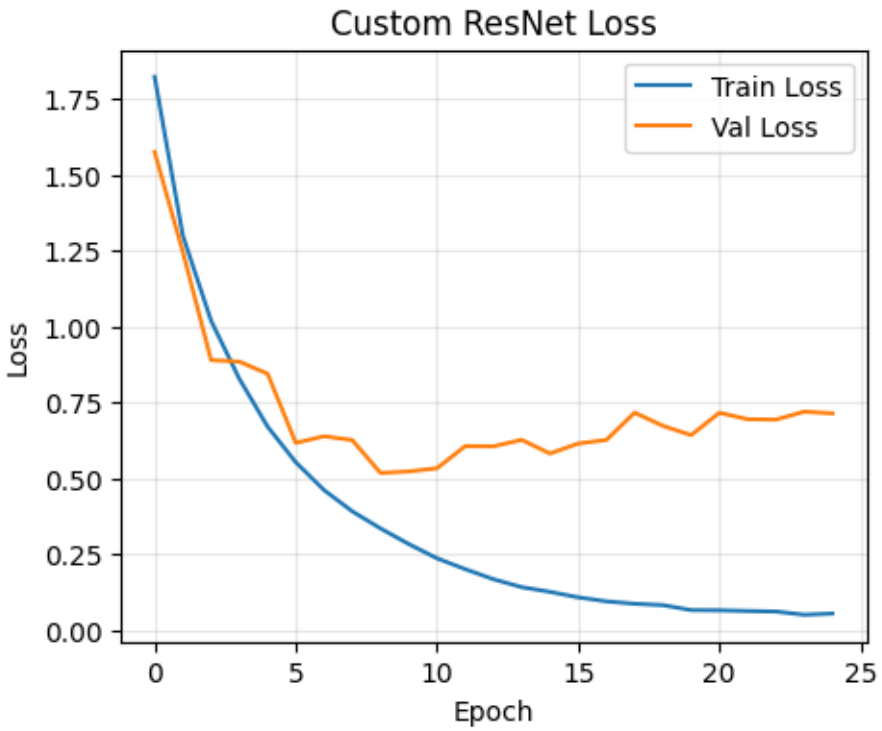
```
Epoch 3/24
----------
train Loss: 0.8283 Acc: 0.7068
Epoch time taken:  14.20264458656311
val Loss: 0.8848 Acc: 0.6865
Epoch time taken:  15.541638135910034

Epoch 4/24
----------
train Loss: 0.6709 Acc: 0.7625
Epoch time taken:  14.216882944107056
val Loss: 0.8452 Acc: 0.7260
Epoch time taken:  15.59026288986206

Epoch 5/24
----------
train Loss: 0.5532 Acc: 0.8060
Epoch time taken:  14.301340341567993
val Loss: 0.6171 Acc: 0.7887
Epoch time taken:  15.68303394317627

Epoch 6/24
----------
train Loss: 0.4617 Acc: 0.8387
Epoch time taken:  18.539992570877075
val Loss: 0.6390 Acc: 0.7880
Epoch time taken:  21.046317100524902

Epoch 7/24
----------
train Loss: 0.3918 Acc: 0.8636
Epoch time taken:  14.325906038284302
val Loss: 0.6263 Acc: 0.7912
Epoch time taken:  15.74747109413147

Epoch 8/24
----------
train Loss: 0.3353 Acc: 0.8831
Epoch time taken:  14.55097246170044
val Loss: 0.5181 Acc: 0.8313
Epoch time taken:  15.996125936508179

Epoch 9/24
----------
train Loss: 0.2839 Acc: 0.9020
Epoch time taken:  14.301948547363281
val Loss: 0.5231 Acc: 0.8323
```

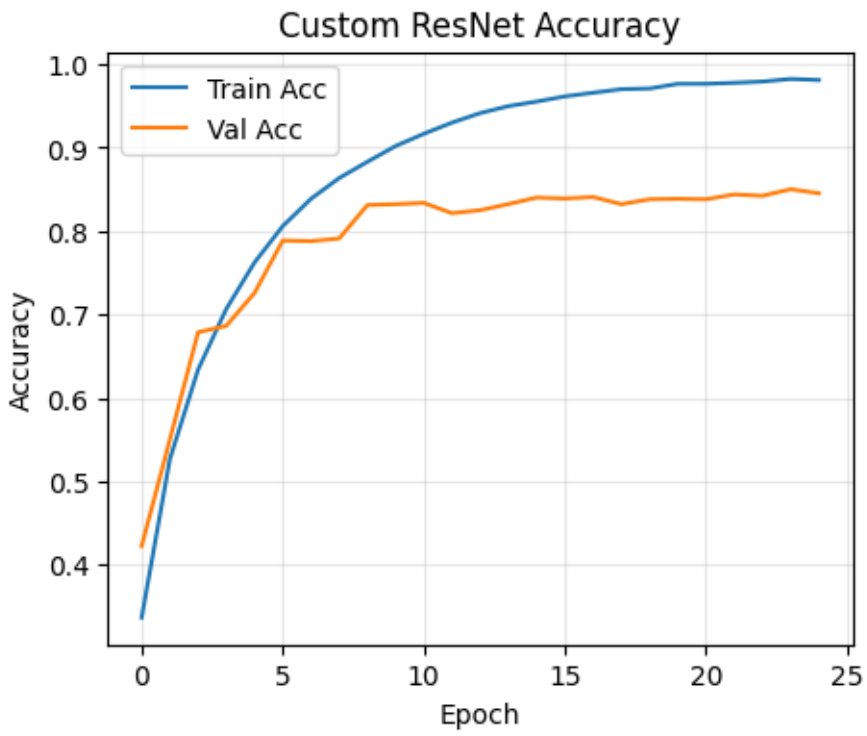
```
Epoch time taken:  15.72651195526123

Epoch 10/24
----------
train Loss: 0.2369 Acc: 0.9167
Epoch time taken:  14.597991466522217
val Loss: 0.5336 Acc: 0.8338
Epoch time taken:  16.011810541152954

Epoch 11/24
----------
train Loss: 0.2012 Acc: 0.9301
Epoch time taken:  14.832728147506714
val Loss: 0.6066 Acc: 0.8215
Epoch time taken:  16.247580766677856

Epoch 12/24
----------
train Loss: 0.1680 Acc: 0.9415
Epoch time taken:  21.409284830093384
val Loss: 0.6058 Acc: 0.8251
Epoch time taken:  24.032949209213257

Epoch 13/24
----------
train Loss: 0.1418 Acc: 0.9499
Epoch time taken:  28.991053819656372
val Loss: 0.6272 Acc: 0.8324
Epoch time taken:  31.579021215438843

Epoch 14/24
----------
train Loss: 0.1265 Acc: 0.9553
Epoch time taken:  29.027026653289795
val Loss: 0.5822 Acc: 0.8402
Epoch time taken:  31.507090091705322

Epoch 15/24
----------
train Loss: 0.1083 Acc: 0.9615
Epoch time taken:  29.62389588356018
val Loss: 0.6150 Acc: 0.8391
Epoch time taken:  32.34744071960449

Epoch 16/24
----------
train Loss: 0.0952 Acc: 0.9659
Epoch time taken:  30.001065492630005
```

```
val Loss: 0.6269 Acc: 0.8409
Epoch time taken:  32.73465061187744

Epoch 17/24
----------
train Loss: 0.0871 Acc: 0.9702
Epoch time taken:  29.035639762878418
val Loss: 0.7169 Acc: 0.8323
Epoch time taken:  31.757688283920288

Epoch 18/24
----------
train Loss: 0.0827 Acc: 0.9710
Epoch time taken:  29.89882493019104
val Loss: 0.6732 Acc: 0.8382
Epoch time taken:  32.45984363555908

Epoch 19/24
----------
train Loss: 0.0664 Acc: 0.9766
Epoch time taken:  29.17538857460022
val Loss: 0.6428 Acc: 0.8387
Epoch time taken:  31.748485803604126

Epoch 20/24
----------
train Loss: 0.0657 Acc: 0.9766
Epoch time taken:  29.717209577560425
val Loss: 0.7165 Acc: 0.8382
Epoch time taken:  32.54269814491272

Epoch 21/24
----------
train Loss: 0.0634 Acc: 0.9778
Epoch time taken:  30.127708911895752
val Loss: 0.6951 Acc: 0.8440
Epoch time taken:  32.89457726478577

Epoch 22/24
----------
train Loss: 0.0617 Acc: 0.9793
Epoch time taken:  29.806729555130005
val Loss: 0.6938 Acc: 0.8423
Epoch time taken:  32.44356632232666

Epoch 23/24
----------
train Loss: 0.0510 Acc: 0.9824
```

```
Epoch time taken:  28.86069965362549
val Loss: 0.7200 Acc: 0.8502
Epoch time taken:  31.608577013015747

Epoch 24/24
----------
train Loss: 0.0549 Acc: 0.9812
Epoch time taken:  29.157089710235596
val Loss: 0.7145 Acc: 0.8454
Epoch time taken:  31.478155612945557

Training complete in 10m 7s
Best val Acc: 0.850200
```

[41]:
```
train_accs=to_float_list(train_accs)
val_accs=to_float_list(val_accs)
train_losses=to_float_list(train_losses)
val_losses=to_float_list(val_losses)
plot_curves(train_accs, val_accs, train_losses, val_losses, "Custom ResNet")
```





0.2 Kaprao-Horapa Dataset

[15]:
```
from torch.utils.data import Dataset, DataLoader
from os import listdir
from PIL import Image


class BasilDataset(Dataset):
    def __init__(self, root_path="vege_dataset/", transform=None):
        # keep root directory
        self.dir = root_path
        # keep transform
        self.transform = transform

        # read all files in kapao and horapa folder
        list_kaprao = listdir(root_path + 'kapao/')
        list_horapa = listdir(root_path + 'horapa/')
        # calculate all number for each class (just in case)
        self.kaprao_len = len(list_kaprao)
        self.horapa_len = len(list_horapa)

        # put the data file path into ids
```

```
        self.ids = [self.dir + 'kapao/' + file for file in list_kaprao if not␣
↪file.startswith('.')]
        self.ids.extend([self.dir + 'horapa/' + file for file in list_horapa if␣
↪not file.startswith('.')])

    def __len__(self):
        # return self.kaprao_len + self.horapa_len
        return len(self.ids)



    def __getitem__(self, i):
        idx = self.ids[i]
        img_file = idx

        # open photo
        pil_img = Image.open(img_file)

        # resize, normalize and convert to pytorch tensor
        if self.transform:
            img = self.transform(pil_img)
        self.pil_img = pil_img

        # get label from file list counter
        if i < self.kaprao_len:
            label = 0
        else:
            label = 1

        return {
            'image': img,
            'label': label,
            'file_name' : img_file,
        }
```

[16]:
```
url = "https://www.dropbox.com/scl/fi/uibcd2grj4ytzrrqyhe6m/vege_dataset.zip?
↪rlkey=mnh861750qb3jhenvxe4l77c5&e=1&dl=1"
zip_path = "vege_dataset.zip"
dataset_root = Path("vege_dataset")

if not dataset_root.exists():
    print(" Downloading dataset from Dropbox...")
    r = requests.get(url)
    with open(zip_path, "wb") as f:
        f.write(r.content)

    print(" Extracting dataset...")
```

```
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(".")
    print(" Extraction complete.")

else:
    print(" Dataset already available at:", dataset_root)
```

```
 Dataset already available at: vege_dataset
```

[17]:
```
root = "vege_dataset/"

transform = transforms.Compose([
    transforms.Resize(32),
    transforms.RandomCrop(28), # CenterCrop
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.
 ↪225])])

dataset = BasilDataset(root, transform)
```

[18]:
```
import matplotlib.pyplot as plt

output_label = ['kaprao', 'horapa']

batch = dataset[0]
image, label, filename = batch['image'], batch['label'], batch['file_name']
pil_img = Image.open(filename)

print(output_label[label])
print(filename)
# (3, 224, 224) pytorch
# pyplot -> (224,224,3)
plt.imshow(pil_img)
plt.show()
```

```
kaprao
vege_dataset/kapao/20220425_213928_resize.jpg
```

[19]:
```
train_loader = DataLoader(dataset, batch_size=64, shuffle=True, pin_memory=True)
```

[21]:
```
resnet = ResNet18(2).to(device)
# Optimizer and loss function
criterion = nn.CrossEntropyLoss()
params_to_update = resnet.parameters()
# Now we'll use Adam optimization
optimizer = optim.Adam(params_to_update, lr=0.01)
```

[23]:
```
n_epochs = 25

loss_history = []
loss_history_epoch = []
accuracy = []

for epoch in range(1, n_epochs + 1):
    epoch_iter = 0                  # the number of training iterations in␣
 ↪current epoch, reset to 0 every epoch
    running_loss = 0
    running_corrects = 0
    for batch in train_loader:
```

```
        image, label, filename = batch['image'], batch['label'],␣
↪batch['file_name']

        epoch_iter += image.shape[0]

        image = image.to(device)
        label = label.to(device)

        # training only
        optimizer.zero_grad()

        output = resnet(image)

        # 0, 1, 0, 0 ---> 0.2, 0.6, 0.1, 0.1
        loss = criterion(output, label)       # training

        # prediction - real use
        _, preds = torch.max(output, 1)

        running_loss += loss.item() * image.size(0)
        running_corrects += torch.sum(preds == label.data)

        loss.backward()       # back propagation -> calculate that how much␣
↪value to update weight
        optimizer.step()      #update weight

        loss_history.append(loss.item() * image.size(0))
        if (epoch_iter % 640 == 0):
            print('{} Loss: {:.4f} Acc: {:.4f}'.format(epoch_iter, loss.item(),␣
↪running_corrects / epoch_iter))

    loss_history_epoch.append(running_loss / epoch_iter)
    accuracy.append(running_corrects / epoch_iter)

    print('Epoch: {} Loss: {:.4f} Acc: {:.4f}'.format(epoch, running_loss /␣
↪epoch_iter, running_corrects / epoch_iter * 100.0))
```

```
640 Loss: 0.7635 Acc: 0.5266
1280 Loss: 0.6168 Acc: 0.6305
Epoch: 1 Loss: 1.0466 Acc: 64.6341
640 Loss: 0.2521 Acc: 0.8438
1280 Loss: 0.3446 Acc: 0.8539
Epoch: 2 Loss: 0.3333 Acc: 85.7245
640 Loss: 0.3273 Acc: 0.8578
1280 Loss: 0.2225 Acc: 0.8734
Epoch: 3 Loss: 0.2985 Acc: 87.3027
640 Loss: 0.5685 Acc: 0.8984
```

```
1280 Loss: 0.2234 Acc: 0.8922
Epoch: 4 Loss: 0.2708 Acc: 88.7374
640 Loss: 0.2712 Acc: 0.9000
1280 Loss: 0.1766 Acc: 0.9000
Epoch: 5 Loss: 0.2403 Acc: 89.9570
640 Loss: 0.1977 Acc: 0.8875
1280 Loss: 0.1896 Acc: 0.9070
Epoch: 6 Loss: 0.2312 Acc: 90.6026
640 Loss: 0.2184 Acc: 0.9094
1280 Loss: 0.0809 Acc: 0.9156
Epoch: 7 Loss: 0.2201 Acc: 91.3917
640 Loss: 0.1716 Acc: 0.9281
1280 Loss: 0.3467 Acc: 0.9195
Epoch: 8 Loss: 0.2187 Acc: 91.6786
640 Loss: 0.0954 Acc: 0.9094
1280 Loss: 0.3729 Acc: 0.9094
Epoch: 9 Loss: 0.2110 Acc: 90.8178
640 Loss: 0.3103 Acc: 0.8953
1280 Loss: 0.2271 Acc: 0.9102
Epoch: 10 Loss: 0.2301 Acc: 91.2482
640 Loss: 0.1765 Acc: 0.9391
1280 Loss: 0.3064 Acc: 0.9305
Epoch: 11 Loss: 0.1789 Acc: 93.1851
640 Loss: 0.0696 Acc: 0.9406
1280 Loss: 0.2306 Acc: 0.9359
Epoch: 12 Loss: 0.1847 Acc: 93.6872
640 Loss: 0.2277 Acc: 0.9313
1280 Loss: 0.1675 Acc: 0.9367
Epoch: 13 Loss: 0.1721 Acc: 93.4003
640 Loss: 0.1515 Acc: 0.9297
1280 Loss: 0.1262 Acc: 0.9328
Epoch: 14 Loss: 0.1603 Acc: 93.4003
640 Loss: 0.1195 Acc: 0.9375
1280 Loss: 0.2496 Acc: 0.9406
Epoch: 15 Loss: 0.1559 Acc: 94.2611
640 Loss: 0.1045 Acc: 0.9563
1280 Loss: 0.2283 Acc: 0.9500
Epoch: 16 Loss: 0.1330 Acc: 94.7633
640 Loss: 0.1918 Acc: 0.9375
1280 Loss: 0.5954 Acc: 0.9344
Epoch: 17 Loss: 0.1713 Acc: 93.5438
640 Loss: 0.1262 Acc: 0.9500
1280 Loss: 0.2051 Acc: 0.9320
Epoch: 18 Loss: 0.1753 Acc: 93.0416
640 Loss: 0.1231 Acc: 0.9422
1280 Loss: 0.0880 Acc: 0.9383
Epoch: 19 Loss: 0.1428 Acc: 94.0459
640 Loss: 0.0706 Acc: 0.9563
```

```
1280 Loss: 0.0822 Acc: 0.9445
Epoch: 20 Loss: 0.1353 Acc: 94.3329
640 Loss: 0.0535 Acc: 0.9563
1280 Loss: 0.1056 Acc: 0.9531
Epoch: 21 Loss: 0.1223 Acc: 95.3372
640 Loss: 0.0595 Acc: 0.9531
1280 Loss: 0.1238 Acc: 0.9547
Epoch: 22 Loss: 0.1240 Acc: 95.4806
640 Loss: 0.2916 Acc: 0.9469
1280 Loss: 0.1058 Acc: 0.9539
Epoch: 23 Loss: 0.1164 Acc: 95.4806
640 Loss: 0.0354 Acc: 0.9500
1280 Loss: 0.1512 Acc: 0.9477
Epoch: 24 Loss: 0.1326 Acc: 94.6915
640 Loss: 0.1883 Acc: 0.9422
1280 Loss: 0.1326 Acc: 0.9422
Epoch: 25 Loss: 0.1537 Acc: 94.2611
```

0.3 Exercise 1

[24]:
```python
import torch
from torch.utils.data import Subset

# Set seed for reproducibility
torch.manual_seed(42)

# Shuffle indices
total_size = len(dataset)
indices = torch.randperm(total_size).tolist()

# Split indices
split = int(0.9 * total_size)
train_indices = indices[:split]
val_indices = indices[split:]

# Create subsets
train_dataset = Subset(dataset, train_indices)
val_dataset = Subset(dataset, val_indices)
```

[25]:
```python
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
ex1_dataloaders = {'train': train_loader, 'val': val_loader}
```

[26]:
```python
resnet = ResNet18().to(device)
# Optimizer and loss function
criterion = nn.CrossEntropyLoss()
params_to_update = resnet.parameters()
```

```
# Now we'll use Adam optimization
optimizer = optim.Adam(params_to_update, lr=0.01)

best_model, train_losses, val_losses, train_accs, val_accs =␣
 ↪train_model(resnet, ex1_dataloaders, criterion, optimizer, 25,␣
 ↪'resnet18_bestsofar')
```

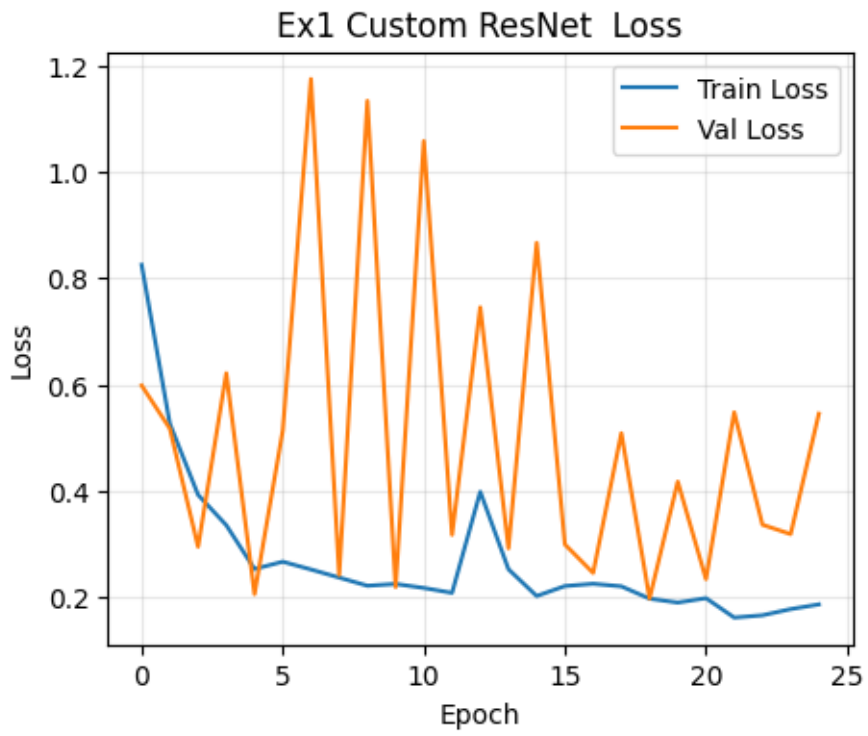
```
Epoch 0/24
----------
train Loss: 0.8250 Acc: 0.6651
Epoch time taken:  4.480274200439453
val Loss: 0.5985 Acc: 0.8429
Epoch time taken:  4.943864822387695

Epoch 1/24
----------
train Loss: 0.5265 Acc: 0.7935
Epoch time taken:  4.444201707839966
val Loss: 0.5173 Acc: 0.7786
Epoch time taken:  4.877918481826782

Epoch 2/24
----------
train Loss: 0.3930 Acc: 0.8341
Epoch time taken:  4.422317266464233
val Loss: 0.2954 Acc: 0.8571
Epoch time taken:  4.862434387207031

Epoch 3/24
----------
train Loss: 0.3360 Acc: 0.8541
Epoch time taken:  4.495402097702026
val Loss: 0.6214 Acc: 0.7500
Epoch time taken:  4.935179948806763

Epoch 4/24
----------
train Loss: 0.2537 Acc: 0.8963
Epoch time taken:  4.384491682052612
val Loss: 0.2064 Acc: 0.9286
Epoch time taken:  4.8294689655303955

Epoch 5/24
----------
train Loss: 0.2674 Acc: 0.8907
Epoch time taken:  4.465078353881836
val Loss: 0.5133 Acc: 0.8143
Epoch time taken:  4.899507284164429
```

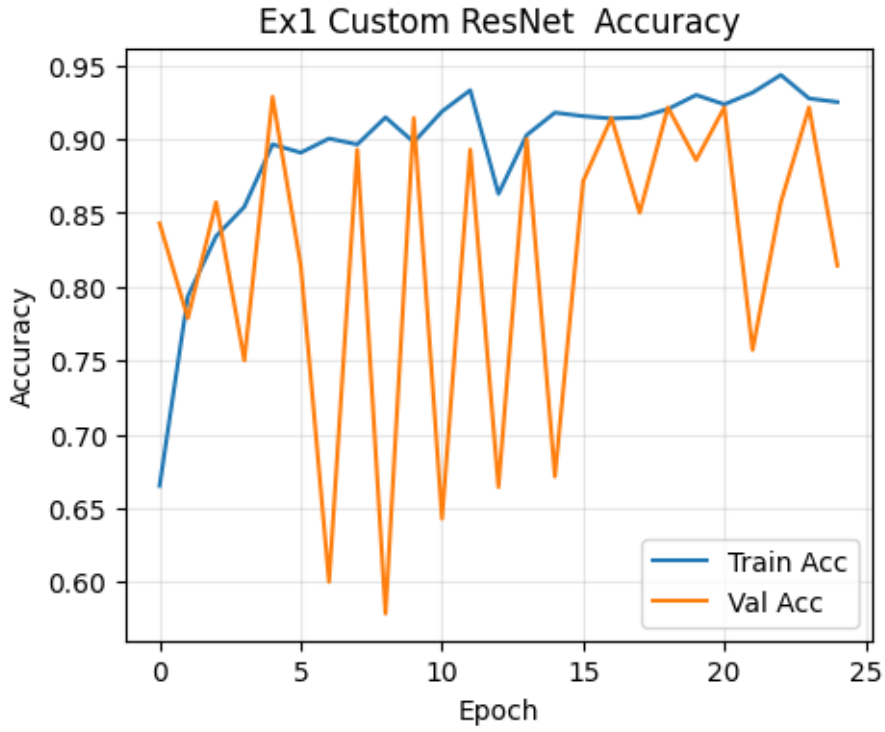
```
Epoch 6/24
----------
train Loss: 0.2528 Acc: 0.9003
Epoch time taken:  4.39209246635437
val Loss: 1.1744 Acc: 0.6000
Epoch time taken:  4.829412937164307

Epoch 7/24
----------
train Loss: 0.2375 Acc: 0.8963
Epoch time taken:  4.432223558425903
val Loss: 0.2424 Acc: 0.8929
Epoch time taken:  4.871108770370483

Epoch 8/24
----------
train Loss: 0.2222 Acc: 0.9147
Epoch time taken:  4.383250951766968
val Loss: 1.1333 Acc: 0.5786
Epoch time taken:  4.8224921226501465

Epoch 9/24
----------
train Loss: 0.2256 Acc: 0.8979
Epoch time taken:  4.410122394561768
val Loss: 0.2197 Acc: 0.9143
Epoch time taken:  4.857949495315552

Epoch 10/24
----------
train Loss: 0.2180 Acc: 0.9187
Epoch time taken:  4.4322509765625
val Loss: 1.0577 Acc: 0.6429
Epoch time taken:  4.875071287155151

Epoch 11/24
----------
train Loss: 0.2088 Acc: 0.9330
Epoch time taken:  4.406672716140747
val Loss: 0.3179 Acc: 0.8929
Epoch time taken:  4.847313642501831

Epoch 12/24
----------
train Loss: 0.3988 Acc: 0.8628
Epoch time taken:  4.435087203979492
val Loss: 0.7453 Acc: 0.6643
```

```
Epoch time taken:  4.890754699707031

Epoch 13/24
----------
train Loss: 0.2532 Acc: 0.9027
Epoch time taken:  4.401139497756958
val Loss: 0.2925 Acc: 0.9000
Epoch time taken:  4.833961725234985

Epoch 14/24
----------
train Loss: 0.2029 Acc: 0.9179
Epoch time taken:  4.392808437347412
val Loss: 0.8665 Acc: 0.6714
Epoch time taken:  4.830170631408691

Epoch 15/24
----------
train Loss: 0.2216 Acc: 0.9155
Epoch time taken:  4.389993667602539
val Loss: 0.2999 Acc: 0.8714
Epoch time taken:  4.848767042160034

Epoch 16/24
----------
train Loss: 0.2259 Acc: 0.9139
Epoch time taken:  4.422107219696045
val Loss: 0.2463 Acc: 0.9143
Epoch time taken:  4.875723838806152

Epoch 17/24
----------
train Loss: 0.2211 Acc: 0.9147
Epoch time taken:  4.424408197402954
val Loss: 0.5089 Acc: 0.8500
Epoch time taken:  4.86761999130249

Epoch 18/24
----------
train Loss: 0.1979 Acc: 0.9203
Epoch time taken:  4.39262318611145
val Loss: 0.1981 Acc: 0.9214
Epoch time taken:  4.830192565917969

Epoch 19/24
----------
train Loss: 0.1906 Acc: 0.9298
Epoch time taken:  4.397557497024536
```

```
val Loss: 0.4179 Acc: 0.8857
Epoch time taken:  4.833327531814575

Epoch 20/24
----------
train Loss: 0.1989 Acc: 0.9234
Epoch time taken:  4.406591176986694
val Loss: 0.2345 Acc: 0.9214
Epoch time taken:  4.845628499984741

Epoch 21/24
----------
train Loss: 0.1624 Acc: 0.9314
Epoch time taken:  4.402268409729004
val Loss: 0.5482 Acc: 0.7571
Epoch time taken:  4.841463327407837

Epoch 22/24
----------
train Loss: 0.1667 Acc: 0.9434
Epoch time taken:  4.3805975914001465
val Loss: 0.3369 Acc: 0.8571
Epoch time taken:  4.81952977180481

Epoch 23/24
----------
train Loss: 0.1783 Acc: 0.9274
Epoch time taken:  4.369910001754761
val Loss: 0.3192 Acc: 0.9214
Epoch time taken:  4.812855243682861

Epoch 24/24
----------
train Loss: 0.1871 Acc: 0.9250
Epoch time taken:  4.4238121509552
val Loss: 0.5454 Acc: 0.8143
Epoch time taken:  4.860166788101196

Training complete in 2m 2s
Best val Acc: 0.928571
```

[27]:
```
train_accs=to_float_list(train_accs)
val_accs=to_float_list(val_accs)
train_losses=to_float_list(train_losses)
val_losses=to_float_list(val_losses)
plot_curves(train_accs, val_accs, train_losses, val_losses, "Ex1 Custom ResNet␣
 ↪")
```


Ex1 Custom ResNet Loss
Train Loss
Val Loss
Loss
Epoch
1.2
1.0
0.8
0.6
0.4
0.2
0
5
10
15
20
25


Ex1 Custom ResNet Accuracy
Train Acc
Val Acc
Accuracy
Epoch
0.95
0.90
0.85
0.80
0.75
0.70
0.65
0.60
0
5
10
15
20
25

0.4 Exercise 2

[12]:
```
class InceptionSEBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        if out_channels % 4 != 0:
            raise ValueError(f"out_channels ({out_channels}) must be divisible␣
 ↪by 4")

        branch_channels = out_channels // 4

        self.branch1 = nn.Conv2d(in_channels, branch_channels, kernel_size=1)

        self.branch3 = nn.Sequential(
            nn.Conv2d(in_channels, branch_channels, kernel_size=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(branch_channels, branch_channels, kernel_size=3,␣
 ↪padding=1)
        )

        self.branch5 = nn.Sequential(
            nn.Conv2d(in_channels, branch_channels, kernel_size=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(branch_channels, branch_channels, kernel_size=5,␣
 ↪padding=2)
        )

        self.pool_branch = nn.Sequential(
            nn.MaxPool2d(kernel_size=3, stride=1, padding=1),
            nn.Conv2d(in_channels, branch_channels, kernel_size=1)
        )

        self.se = SELayer(out_channels)

    def forward(self, x):
        b1 = self.branch1(x)
        b3 = self.branch3(x)
        b5 = self.branch5(x)
        bp = self.pool_branch(x)

        out = torch.cat([b1, b3, b5, bp], dim=1)
        out = self.se(out)
        return out
```

[13]:
```
class InceptionSENet(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
```

```
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True)
        )
        self.block1 = InceptionSEBlock(64, 128)
        self.block2 = InceptionSEBlock(128, 256)
        self.block3 = InceptionSEBlock(256, 256)
        self.pool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(256, num_classes)

    def forward(self, x):
        x = self.stem(x)
        x = self.block1(x)
        x = self.block2(x)
        x = self.block3(x)
        x = self.pool(x).view(x.size(0), -1)
        return self.fc(x)
```

[19]:
```
train_preprocess = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.49139968, 0.48215827, 0.44653124), (0.24703233, 0.
 ↪24348505, 0.26158768))])

eval_preprocess = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.49139968, 0.48215827, 0.44653124), (0.24703233, 0.
 ↪24348505, 0.26158768))])

# Download CIFAR-10 and split into training, validation, and test sets.
# The copy of the training dataset after the split allows us to keep
# the same training/validation split of the original training set but
# apply different transforms to the training set and validation set.

full_train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                                  download=True)

train_dataset, val_dataset = torch.utils.data.random_split(full_train_dataset,␣
 ↪[40000, 10000])
train_dataset.dataset = copy(full_train_dataset)
train_dataset.dataset.transform = train_preprocess
val_dataset.dataset.transform = eval_preprocess

test_dataset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                            download=True,␣
 ↪transform=eval_preprocess)
```

```
# DataLoaders for the three datasets

BATCH_SIZE=128
NUM_WORKERS=4

train_dataloader = torch.utils.data.DataLoader(train_dataset,␣
 ↪batch_size=BATCH_SIZE,
                                               shuffle=True,␣
 ↪num_workers=NUM_WORKERS)
val_dataloader = torch.utils.data.DataLoader(val_dataset, batch_size=BATCH_SIZE,
                                               shuffle=False,␣
 ↪num_workers=NUM_WORKERS)
test_dataloader = torch.utils.data.DataLoader(test_dataset,␣
 ↪batch_size=BATCH_SIZE,
                                               shuffle=False,␣
 ↪num_workers=NUM_WORKERS)

dataloaders = {'train': train_dataloader, 'val': val_dataloader}
```

[21]:
```
inceptse_model = InceptionSENet(num_classes=10).to(device)
params_to_update = inceptse_model.parameters()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(params_to_update, lr=0.001)

model, train_losses, val_losses, train_accs, val_accs = train_model(
    inceptse_model, dataloaders, criterion, optimizer,
    num_epochs=25,
    weights_name='inception_se_cifar10',
    is_inception=False
)
```

```
Epoch 0/24
----------
train Loss: 1.7943 Acc: 0.3122
Epoch time taken:  19.345677375793457
val Loss: 1.5892 Acc: 0.4013
Epoch time taken:  21.05475616455078

Epoch 1/24
----------
train Loss: 1.5029 Acc: 0.4387
Epoch time taken:  19.37024164199829
val Loss: 1.4123 Acc: 0.4820
Epoch time taken:  21.04000186920166

Epoch 2/24
```

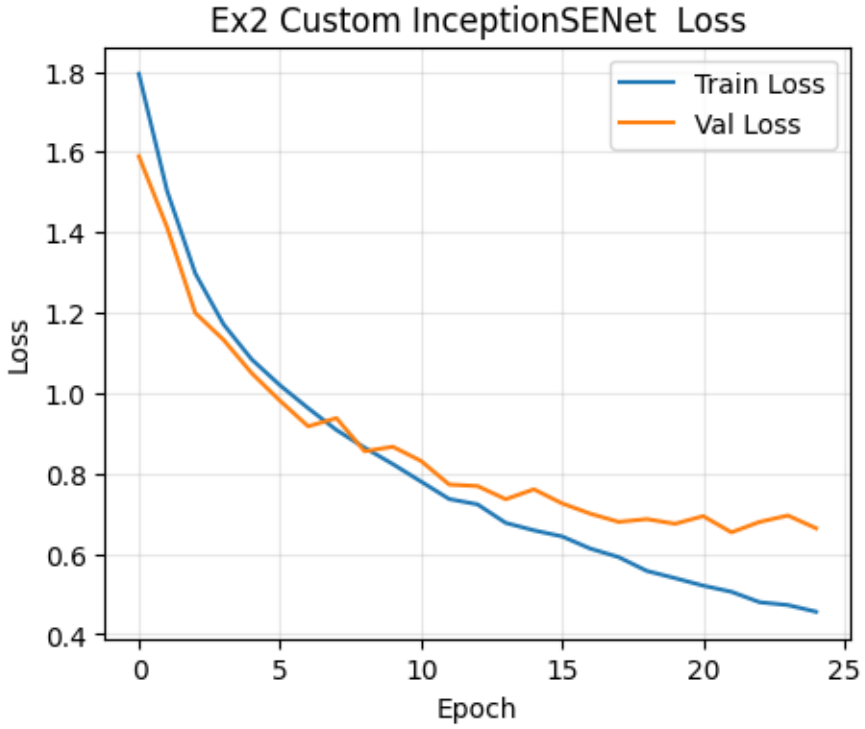
```
----------
train Loss: 1.2976 Acc: 0.5245
Epoch time taken:  19.959463596343994
val Loss: 1.1992 Acc: 0.5644
Epoch time taken:  21.703158855438232

Epoch 3/24
----------
train Loss: 1.1711 Acc: 0.5720
Epoch time taken:  20.547998905181885
val Loss: 1.1325 Acc: 0.5974
Epoch time taken:  22.34153389930725

Epoch 4/24
----------
train Loss: 1.0838 Acc: 0.6037
Epoch time taken:  20.39380931854248
val Loss: 1.0496 Acc: 0.6179
Epoch time taken:  22.161308765411377

Epoch 5/24
----------
train Loss: 1.0195 Acc: 0.6303
Epoch time taken:  20.365654945373535
val Loss: 0.9811 Acc: 0.6413
Epoch time taken:  22.21278405189514

Epoch 6/24
----------
train Loss: 0.9624 Acc: 0.6520
Epoch time taken:  20.10032629966736
val Loss: 0.9173 Acc: 0.6663
Epoch time taken:  21.787353515625

Epoch 7/24
----------
train Loss: 0.9084 Acc: 0.6726
Epoch time taken:  19.82189106941223
val Loss: 0.9378 Acc: 0.6613
Epoch time taken:  21.512434244155884

Epoch 8/24
----------
train Loss: 0.8641 Acc: 0.6895
Epoch time taken:  19.808974266052246
val Loss: 0.8553 Acc: 0.6961
Epoch time taken:  21.51094698905945
```

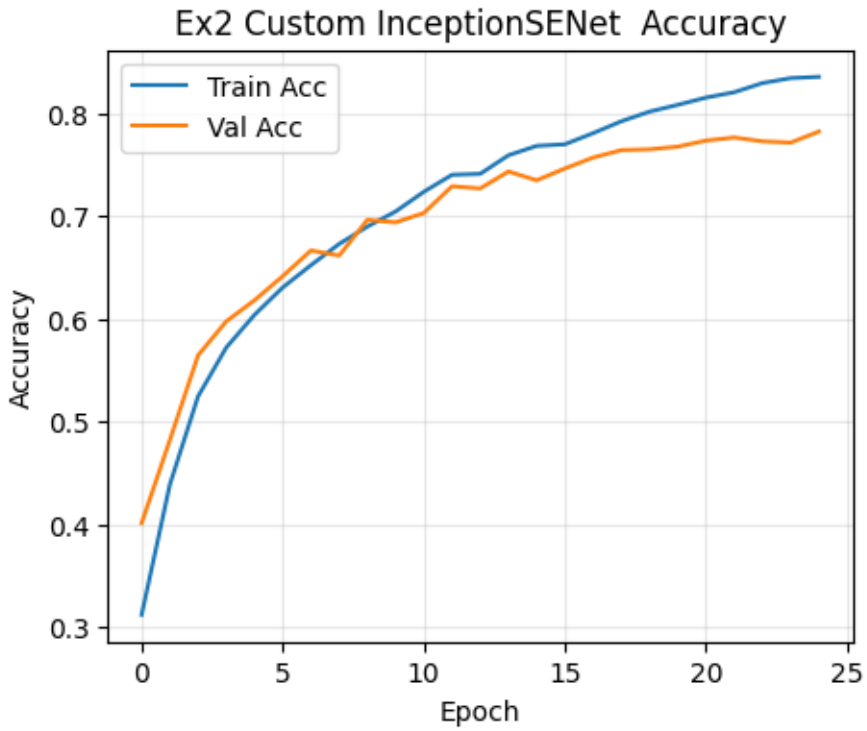
```
Epoch 9/24
----------
train Loss: 0.8234 Acc: 0.7042
Epoch time taken:  19.800461530685425
val Loss: 0.8666 Acc: 0.6936
Epoch time taken:  21.484449863433838

Epoch 10/24
----------
train Loss: 0.7799 Acc: 0.7233
Epoch time taken:  19.853983879089355
val Loss: 0.8311 Acc: 0.7027
Epoch time taken:  21.531352281570435

Epoch 11/24
----------
train Loss: 0.7364 Acc: 0.7399
Epoch time taken:  19.738133192062378
val Loss: 0.7724 Acc: 0.7287
Epoch time taken:  21.429063081741333

Epoch 12/24
----------
train Loss: 0.7232 Acc: 0.7408
Epoch time taken:  19.711485147476196
val Loss: 0.7688 Acc: 0.7266
Epoch time taken:  21.434080362319946

Epoch 13/24
----------
train Loss: 0.6771 Acc: 0.7588
Epoch time taken:  19.79799175262451
val Loss: 0.7360 Acc: 0.7431
Epoch time taken:  21.463789463043213

Epoch 14/24
----------
train Loss: 0.6586 Acc: 0.7680
Epoch time taken:  19.787254810333252
val Loss: 0.7607 Acc: 0.7345
Epoch time taken:  21.50852942466736

Epoch 15/24
----------
train Loss: 0.6434 Acc: 0.7695
Epoch time taken:  19.80423641204834
val Loss: 0.7255 Acc: 0.7460
Epoch time taken:  21.491894006729126
```

```
Epoch 16/24
----------
train Loss: 0.6131 Acc: 0.7803
Epoch time taken:  19.797687292099
val Loss: 0.7000 Acc: 0.7566
Epoch time taken:  21.48466181755066

Epoch 17/24
----------
train Loss: 0.5920 Acc: 0.7919
Epoch time taken:  19.716026306152344
val Loss: 0.6797 Acc: 0.7638
Epoch time taken:  21.415921211242676

Epoch 18/24
----------
train Loss: 0.5576 Acc: 0.8013
Epoch time taken:  19.74843168258667
val Loss: 0.6865 Acc: 0.7646
Epoch time taken:  21.435365200042725

Epoch 19/24
----------
train Loss: 0.5397 Acc: 0.8079
Epoch time taken:  19.670695304870605
val Loss: 0.6749 Acc: 0.7672
Epoch time taken:  21.353123426437378

Epoch 20/24
----------
train Loss: 0.5210 Acc: 0.8149
Epoch time taken:  19.76027750968933
val Loss: 0.6939 Acc: 0.7731
Epoch time taken:  21.464502811431885

Epoch 21/24
----------
train Loss: 0.5057 Acc: 0.8201
Epoch time taken:  19.76600170135498
val Loss: 0.6536 Acc: 0.7761
Epoch time taken:  21.478313446044922

Epoch 22/24
----------
train Loss: 0.4797 Acc: 0.8290
Epoch time taken:  19.722333431243896
val Loss: 0.6796 Acc: 0.7724
```

```
Epoch time taken:  21.400831699371338

Epoch 23/24
----------
train Loss: 0.4727 Acc: 0.8338
Epoch time taken:  19.80682396888733
val Loss: 0.6955 Acc: 0.7711
Epoch time taken:  21.495614528656006

Epoch 24/24
----------
train Loss: 0.4561 Acc: 0.8351
Epoch time taken:  19.794129133224487
val Loss: 0.6637 Acc: 0.7819
Epoch time taken:  21.467000722885132

Training complete in 8m 59s
Best val Acc: 0.781900
```

[22]:
```
train_accs=to_float_list(train_accs)
val_accs=to_float_list(val_accs)
train_losses=to_float_list(train_losses)
val_losses=to_float_list(val_losses)
plot_curves(train_accs, val_accs, train_losses, val_losses, "Ex2 Custom␣
 ↪InceptionSENet ")
```





0.5 Exercise 3

[11]:
```
!mkdir -p TrashClassification

# Download the dataset ZIP directly using curl
!curl -L -o TrashClassification/finaldata.zip \
  https://www.kaggle.com/api/v1/datasets/download/surabhisathish/finaldata
```

```
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100  103M  100  103M    0     0  9320k      0  0:00:11  0:00:11 --:--:-- 11.0M
```

[12]:
```
import zipfile, os

zip_path = "TrashClassification/finaldata.zip"
extract_path = "TrashClassification"

if os.path.exists(zip_path):
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall(extract_path)
```

```
    print("Dataset extracted to:", extract_path)
else:
    print("ZIP file not found.")
```

```
Dataset extracted to: TrashClassification
```

[14]:
```
base_dir = "TrashClassification/rubbish-data"

train_dir = os.path.join(base_dir, "train")
val_dir   = os.path.join(base_dir, "val")
test_dir  = os.path.join(base_dir, "test")
```

[15]:
```
mean = (0.485, 0.456, 0.406)
std  = (0.229, 0.224, 0.225)

train_tfms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])

val_test_tfms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
])
```

[16]:
```
train_dataset = datasets.ImageFolder(root=train_dir, transform=train_tfms)
val_dataset   = datasets.ImageFolder(root=val_dir,   transform=val_test_tfms)
test_dataset  = datasets.ImageFolder(root=test_dir,  transform=val_test_tfms)

print(f" Train size: {len(train_dataset)} | Val size: {len(val_dataset)} | Test␣
 ↪size: {len(test_dataset)}")
print(f" Number of classes: {len(train_dataset.classes)}")
print(f"Classes: {train_dataset.classes}")
```

```
 Train size: 7975 | Val size: 1050 | Test size: 1113
 Number of classes: 10
Classes: ['battery', 'biological', 'brown-glass', 'cardboard', 'green-glass',
'metal', 'paper', 'plastic', 'trash', 'white-glass']
```

[19]:
```
BATCH_SIZE = 16
NUM_WORKERS = 2

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, ␣
 ↪num_workers=NUM_WORKERS, pin_memory=True)
```

```
val_loader   = DataLoader(val_dataset,   batch_size=BATCH_SIZE, shuffle=False,␣
 ↪num_workers=NUM_WORKERS, pin_memory=True)
test_loader  = DataLoader(test_dataset,  batch_size=BATCH_SIZE, shuffle=False,␣
 ↪num_workers=NUM_WORKERS, pin_memory=True)

dataloaders = {'train': train_loader, 'val': val_loader}
```

[20]:
```
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

output_dir = "output_models"
os.makedirs(output_dir, exist_ok=True)

def train_model2(model, dataloaders, criterion, optimizer, num_epochs=25,␣
 ↪weights_name='weight_save', is_inception=False):
    import torch
    import time
    from copy import deepcopy
    from torch.cuda.amp import autocast, GradScaler


    since = time.time()
    train_losses, val_losses = [], []
    train_accs, val_accs = [], []

    best_model_wts = deepcopy(model.state_dict())
    best_acc = 0.0
    scaler = GradScaler()  # For mixed precision

    for epoch in range(num_epochs):
        epoch_start = time.time()
        print(f'Epoch {epoch}/{num_epochs - 1}\n{"-" * 10}')

        for phase in ['train', 'val']:
            model.train() if phase == 'train' else model.eval()
            running_loss = 0.0
            running_corrects = 0

            for batch in dataloaders[phase]:
                if isinstance(batch, dict):
                    inputs = batch['image']
                    labels = batch['label']
                else:
                    inputs, labels = batch

                inputs = inputs.to(device, non_blocking=True)
                labels = labels.to(device, non_blocking=True)
```

```
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    with autocast():  # Mixed precision
                        if is_inception and phase == 'train':
                            outputs, aux_outputs = model(inputs)
                            loss1 = criterion(outputs, labels)
                            loss2 = criterion(aux_outputs, labels)
                            loss = loss1 + 0.4 * loss2
                        else:
                            outputs = model(inputs)
                            loss = criterion(outputs, labels)

                        _, preds = torch.max(outputs, 1)

                        if phase == 'train':
                            scaler.scale(loss).backward()
                            scaler.step(optimizer)
                            scaler.update()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data)

                # Free memory
                del inputs, labels, outputs, preds, loss
                torch.cuda.empty_cache()

            epoch_loss = running_loss / len(dataloaders[phase].dataset)
            epoch_acc = running_corrects.double() / len(dataloaders[phase].
↪dataset)
            elapsed_epoch = time.time() - epoch_start

            print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')
            print("Epoch time taken:", elapsed_epoch)

            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = deepcopy(model.state_dict())
                torch.save(model.state_dict(), os.path.join(output_dir,␣
↪weights_name + ".pth"))

            if phase == 'train':
                train_losses.append(epoch_loss)
                train_accs.append(epoch_acc)
            else:
                val_losses.append(epoch_loss)
```

```
                val_accs.append(epoch_acc)

        print()

    time_elapsed = time.time() - since
    print(f'Training complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.
 ↪0f}s')
    print(f'Best val Acc: {best_acc:.4f}')

    model.load_state_dict(best_model_wts)
    return model, train_losses, val_losses, train_accs, val_accs
```

[21]:
```
inceptse_model = InceptionSENet(num_classes=10).to(device)
params_to_update = inceptse_model.parameters()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(params_to_update, lr=0.001)

model, train_losses, val_losses, train_accs, val_accs = train_model2(
    inceptse_model, dataloaders, criterion, optimizer,
    num_epochs=25,
    weights_name='inception_se_cifar10',
    is_inception=False
)
```

```
Epoch 0/24
----------
train Loss: 1.6303 Acc: 0.4193
Epoch time taken: 147.5932013988495
val Loss: 1.2111 Acc: 0.5714
Epoch time taken: 153.2510974407196

Epoch 1/24
----------
train Loss: 1.2702 Acc: 0.5564
Epoch time taken: 150.75545263290405
val Loss: 1.0622 Acc: 0.6276
Epoch time taken: 156.4108748435974

Epoch 2/24
----------
train Loss: 1.1400 Acc: 0.6063
Epoch time taken: 151.3446900844574
val Loss: 0.9941 Acc: 0.6410
Epoch time taken: 157.00489115715027

Epoch 3/24
----------
train Loss: 1.0646 Acc: 0.6290
```

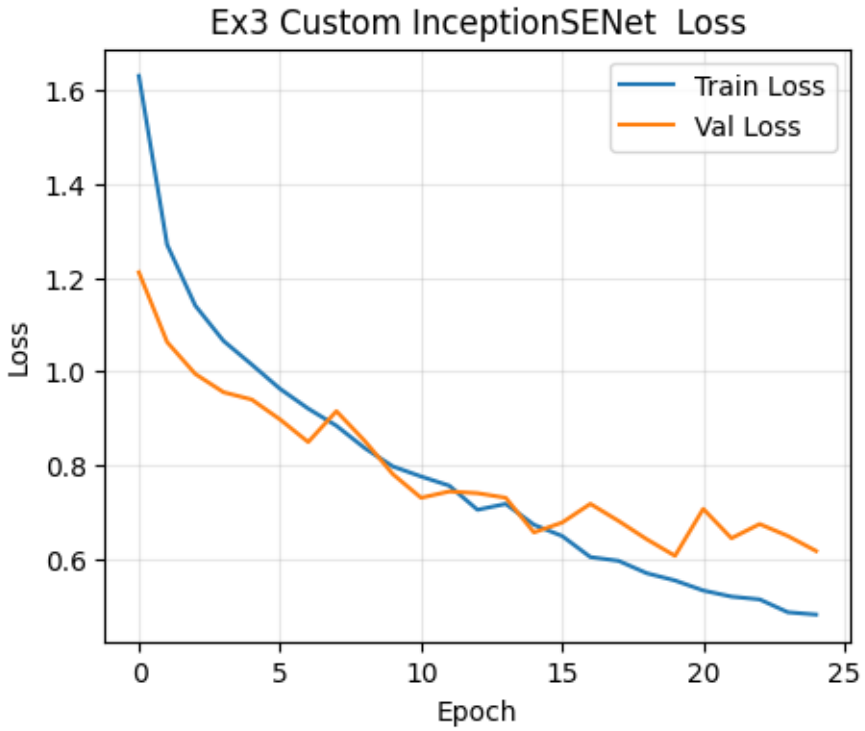
```
Epoch time taken: 151.046972990036
val Loss: 0.9552 Acc: 0.6590
Epoch time taken: 156.7180736064911

Epoch 4/24
----------
train Loss: 1.0142 Acc: 0.6475
Epoch time taken: 150.5428910255432
val Loss: 0.9395 Acc: 0.6790
Epoch time taken: 156.22001814842224

Epoch 5/24
----------
train Loss: 0.9625 Acc: 0.6692
Epoch time taken: 150.26923966407776
val Loss: 0.8971 Acc: 0.6962
Epoch time taken: 155.8871512413025

Epoch 6/24
----------
train Loss: 0.9202 Acc: 0.6750
Epoch time taken: 150.29883694648743
val Loss: 0.8488 Acc: 0.7200
Epoch time taken: 155.92800068855286

Epoch 7/24
----------
train Loss: 0.8836 Acc: 0.6905
Epoch time taken: 150.0375542640686
val Loss: 0.9147 Acc: 0.6848
Epoch time taken: 155.65545058250427

Epoch 8/24
----------
train Loss: 0.8360 Acc: 0.7124
Epoch time taken: 148.9716408252716
val Loss: 0.8515 Acc: 0.7219
Epoch time taken: 154.60091471672058

Epoch 9/24
----------
train Loss: 0.7967 Acc: 0.7214
Epoch time taken: 149.4933705329895
val Loss: 0.7802 Acc: 0.7390
Epoch time taken: 155.06534671783447

Epoch 10/24
----------
```

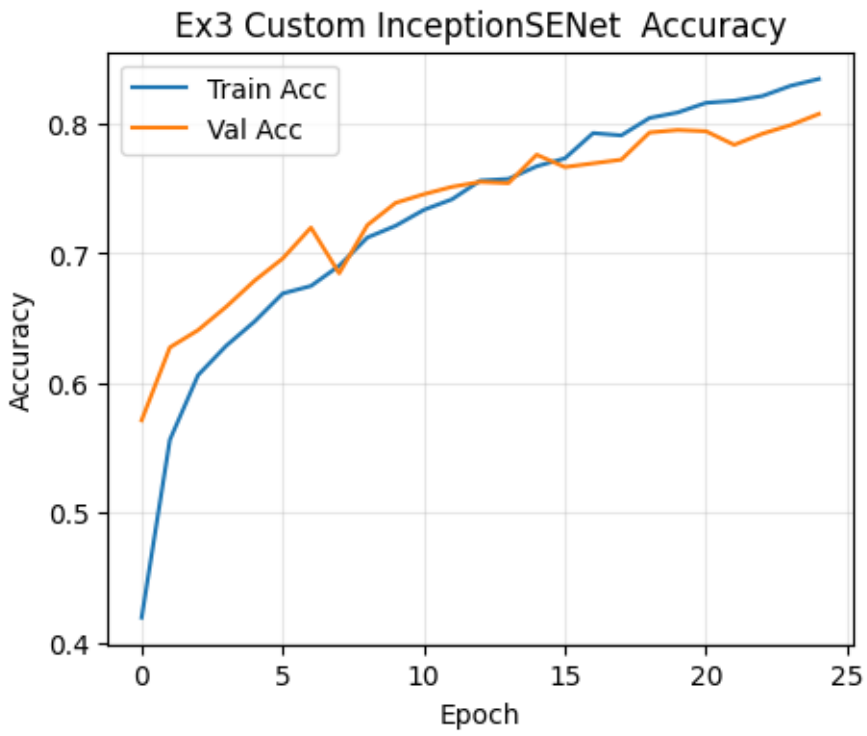
```
train Loss: 0.7753 Acc: 0.7337
Epoch time taken: 149.2015118598938
val Loss: 0.7298 Acc: 0.7457
Epoch time taken: 154.8262095451355

Epoch 11/24
----------
train Loss: 0.7556 Acc: 0.7418
Epoch time taken: 149.32865238189697
val Loss: 0.7433 Acc: 0.7514
Epoch time taken: 155.05098176002502

Epoch 12/24
----------
train Loss: 0.7042 Acc: 0.7562
Epoch time taken: 149.5800929069519
val Loss: 0.7396 Acc: 0.7552
Epoch time taken: 155.15567803382874

Epoch 13/24
----------
train Loss: 0.7167 Acc: 0.7575
Epoch time taken: 148.81891417503357
val Loss: 0.7298 Acc: 0.7543
Epoch time taken: 154.4576394557953

Epoch 14/24
----------
train Loss: 0.6722 Acc: 0.7673
Epoch time taken: 149.0478687286377
val Loss: 0.6559 Acc: 0.7762
Epoch time taken: 154.75188851356506

Epoch 15/24
----------
train Loss: 0.6482 Acc: 0.7734
Epoch time taken: 149.1386275291443
val Loss: 0.6769 Acc: 0.7667
Epoch time taken: 154.82644081115723

Epoch 16/24
----------
train Loss: 0.6032 Acc: 0.7929
Epoch time taken: 150.32725954055786
val Loss: 0.7169 Acc: 0.7695
Epoch time taken: 156.00330257415771

Epoch 17/24
```

```
----------
train Loss: 0.5952 Acc: 0.7911
Epoch time taken: 149.57118725776672
val Loss: 0.6803 Acc: 0.7724
Epoch time taken: 155.26978421211243

Epoch 18/24
----------
train Loss: 0.5687 Acc: 0.8045
Epoch time taken: 149.66380643844604
val Loss: 0.6412 Acc: 0.7933
Epoch time taken: 155.29446387290955

Epoch 19/24
----------
train Loss: 0.5533 Acc: 0.8088
Epoch time taken: 149.46376657485962
val Loss: 0.6059 Acc: 0.7952
Epoch time taken: 155.05318427085876

Epoch 20/24
----------
train Loss: 0.5319 Acc: 0.8162
Epoch time taken: 149.3574516773224
val Loss: 0.7062 Acc: 0.7943
Epoch time taken: 154.9231026172638

Epoch 21/24
----------
train Loss: 0.5188 Acc: 0.8178
Epoch time taken: 149.82845187187195
val Loss: 0.6435 Acc: 0.7838
Epoch time taken: 155.4908046722412

Epoch 22/24
----------
train Loss: 0.5129 Acc: 0.8214
Epoch time taken: 149.5142433643341
val Loss: 0.6737 Acc: 0.7924
Epoch time taken: 155.1487066745758

Epoch 23/24
----------
train Loss: 0.4854 Acc: 0.8293
Epoch time taken: 149.7236454486847
val Loss: 0.6482 Acc: 0.7990
Epoch time taken: 155.36259722709656
```

```
Epoch 24/24
----------
train Loss: 0.4805 Acc: 0.8346
Epoch time taken: 149.86589932441711
val Loss: 0.6162 Acc: 0.8076
Epoch time taken: 155.46267247200012

Training complete in 64m 44s
Best val Acc: 0.8076
```

[23]:
```
train_accs=to_float_list(train_accs)
val_accs=to_float_list(val_accs)
train_losses=to_float_list(train_losses)
val_losses=to_float_list(val_losses)

plot_curves(train_accs, val_accs, train_losses, val_losses, "Ex3 Custom␣
 ↪InceptionSENet ")
```

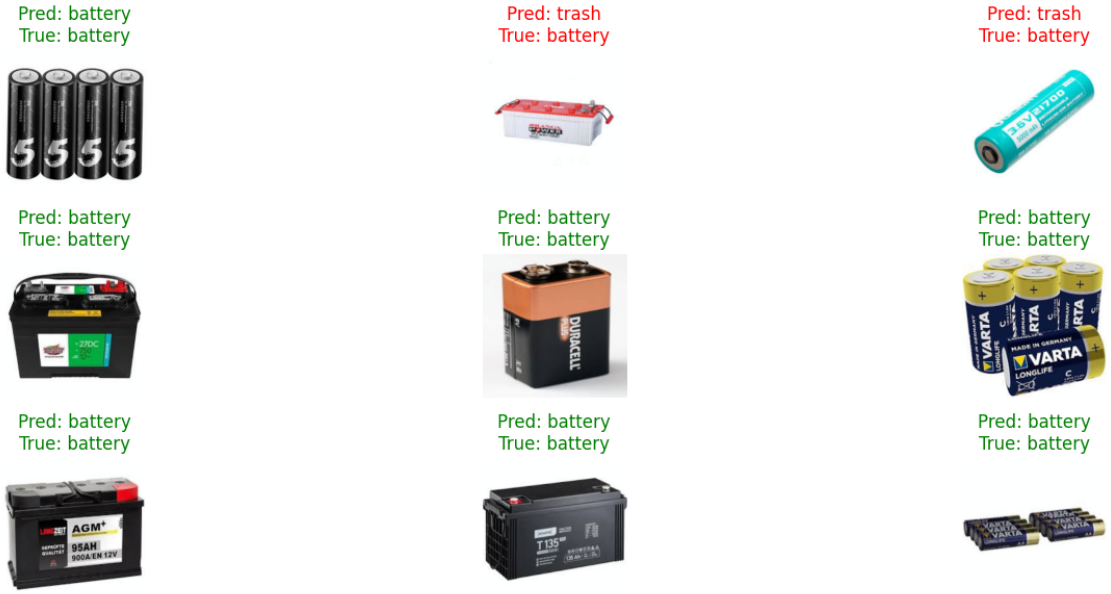




[27]:
```python
def show_predictions(model, dataloader, class_names, device, num_images=6):
    """
    Display sample predictions with image and labels.

    Args:
        model: trained model (in eval mode)
        dataloader: DataLoader (e.g. test_loader)
        class_names: list of class names (dataset.classes)
        device: torch.device('cuda' or 'cpu')
        num_images: number of images to display
    """
    model.eval()
    images_shown = 0
    plt.figure(figsize=(14, 8))

    with torch.no_grad():
        for images, labels in dataloader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            for j in range(images.size(0)):
                images_shown += 1
```

```
                ax = plt.subplot(num_images//3 + 1, 3, images_shown)
                ax.axis('off')

                # Denormalize for display
                img = images[j].cpu().permute(1, 2, 0).numpy()
                mean = [0.485, 0.456, 0.406]
                std = [0.229, 0.224, 0.225]
                img = std * img + mean
                img = torch.clip(torch.tensor(img), 0, 1).numpy()

                ax.imshow(img)
                true_label = class_names[labels[j].item()]
                pred_label = class_names[preds[j].item()]

                color = "green" if pred_label == true_label else "red"
                ax.set_title(f"Pred: {pred_label}\nTrue: {true_label}",␣
↪color=color)

                if images_shown == num_images:
                    plt.tight_layout()
                    plt.show()
                    return
```

[28]:
```
show_predictions(model=inceptse_model,
                 dataloader=test_loader,
                 class_names=train_dataset.classes,
                 device=device,
                 num_images=9)
```



[29]:
```
resnet = ResNet18().to(device)
# Optimizer and loss function
criterion = nn.CrossEntropyLoss()
params_to_update = resnet.parameters()
# Now we'll use Adam optimization
optimizer = optim.Adam(params_to_update, lr=0.01)

best_model, train_losses, val_losses, train_accs, val_accs =␣
 ↪train_model2(resnet, dataloaders, criterion, optimizer, 25, 'resnet18_ex3')
```

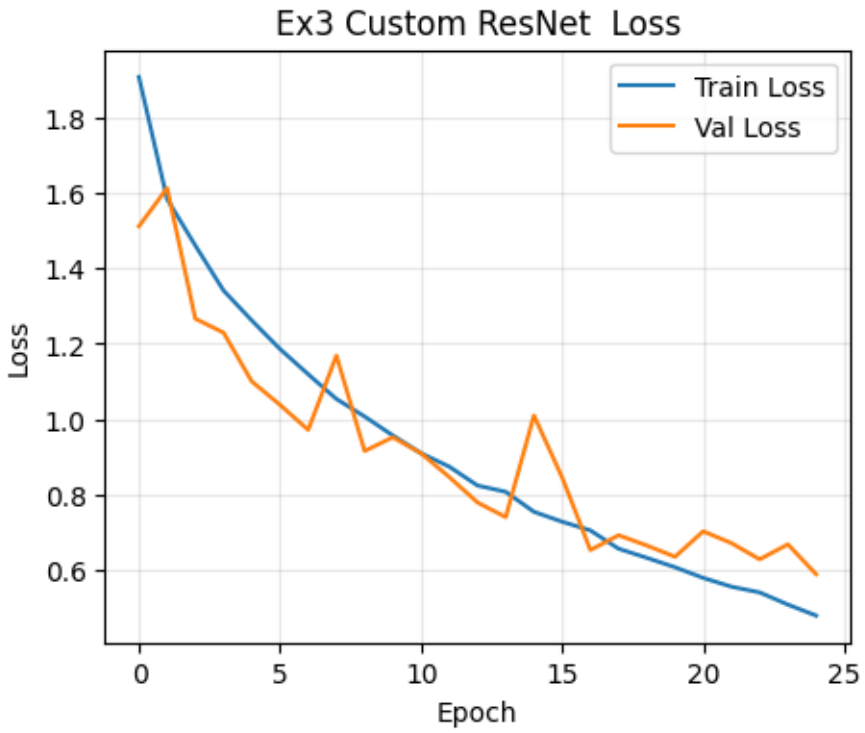
```
Epoch 0/24
----------
train Loss: 1.9078 Acc: 0.3487
Epoch time taken: 70.84894680976868
val Loss: 1.5115 Acc: 0.4676
Epoch time taken: 73.69174432754517

Epoch 1/24
----------
train Loss: 1.5838 Acc: 0.4559
Epoch time taken: 75.83431267738342
val Loss: 1.6130 Acc: 0.4838
Epoch time taken: 78.72165012359619

Epoch 2/24
----------
train Loss: 1.4608 Acc: 0.5076
Epoch time taken: 73.80965948104858
val Loss: 1.2667 Acc: 0.5600
Epoch time taken: 76.6917130947113

Epoch 3/24
----------
train Loss: 1.3413 Acc: 0.5614
Epoch time taken: 75.72857809066772
val Loss: 1.2297 Acc: 0.6152
Epoch time taken: 78.61885666847229

Epoch 4/24
----------
train Loss: 1.2621 Acc: 0.5804
Epoch time taken: 74.28998327255249
val Loss: 1.1004 Acc: 0.6467
Epoch time taken: 77.18762397766113

Epoch 5/24
```

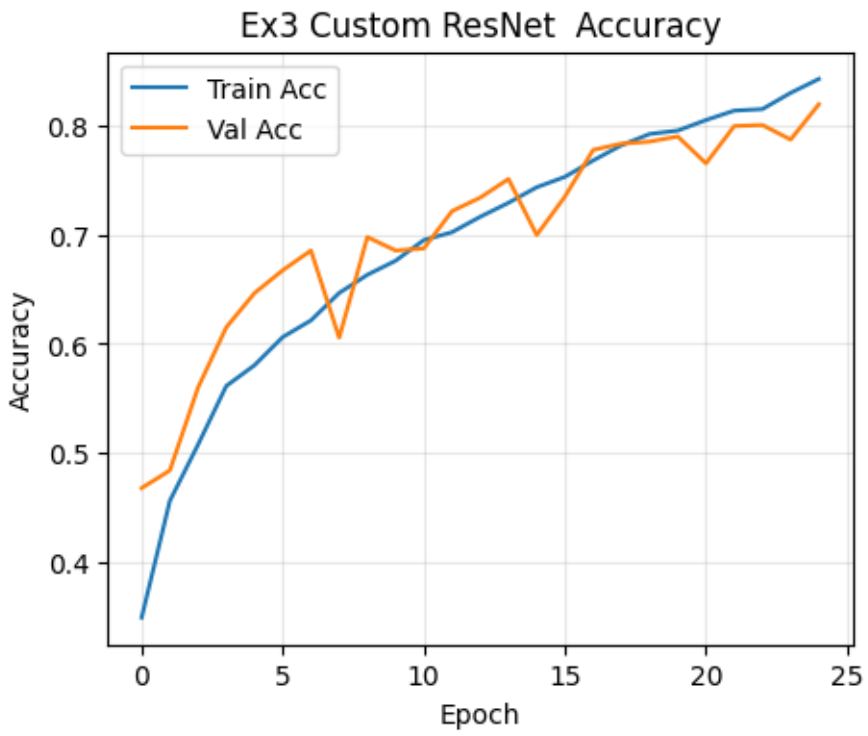
```
----------
train Loss: 1.1863 Acc: 0.6063
Epoch time taken: 77.68592715263367
val Loss: 1.0381 Acc: 0.6676
Epoch time taken: 80.54620885848999

Epoch 6/24
----------
train Loss: 1.1194 Acc: 0.6216
Epoch time taken: 74.14716339111328
val Loss: 0.9719 Acc: 0.6857
Epoch time taken: 77.00047135353088

Epoch 7/24
----------
train Loss: 1.0537 Acc: 0.6468
Epoch time taken: 77.05533576011658
val Loss: 1.1689 Acc: 0.6057
Epoch time taken: 79.95370888710022

Epoch 8/24
----------
train Loss: 1.0073 Acc: 0.6634
Epoch time taken: 76.91934657096863
val Loss: 0.9157 Acc: 0.6981
Epoch time taken: 79.8054780960083

Epoch 9/24
----------
train Loss: 0.9569 Acc: 0.6764
Epoch time taken: 76.21314907073975
val Loss: 0.9523 Acc: 0.6857
Epoch time taken: 79.13584804534912

Epoch 10/24
----------
train Loss: 0.9098 Acc: 0.6953
Epoch time taken: 76.158860206604
val Loss: 0.9095 Acc: 0.6876
Epoch time taken: 79.09777021408081

Epoch 11/24
----------
train Loss: 0.8742 Acc: 0.7026
Epoch time taken: 76.16892194747925
val Loss: 0.8467 Acc: 0.7219
Epoch time taken: 79.09660720825195
```

```
Epoch 12/24
----------
train Loss: 0.8243 Acc: 0.7167
Epoch time taken: 74.70258021354675
val Loss: 0.7793 Acc: 0.7343
Epoch time taken: 77.57227754592896

Epoch 13/24
----------
train Loss: 0.8076 Acc: 0.7295
Epoch time taken: 73.87676644325256
val Loss: 0.7403 Acc: 0.7514
Epoch time taken: 76.75595283508301

Epoch 14/24
----------
train Loss: 0.7546 Acc: 0.7438
Epoch time taken: 75.95701766014099
val Loss: 1.0100 Acc: 0.7000
Epoch time taken: 78.85725903511047

Epoch 15/24
----------
train Loss: 0.7282 Acc: 0.7534
Epoch time taken: 75.61034035682678
val Loss: 0.8457 Acc: 0.7352
Epoch time taken: 78.48707675933838

Epoch 16/24
----------
train Loss: 0.7053 Acc: 0.7684
Epoch time taken: 75.56994867324829
val Loss: 0.6531 Acc: 0.7781
Epoch time taken: 78.41872930526733

Epoch 17/24
----------
train Loss: 0.6566 Acc: 0.7823
Epoch time taken: 73.54458570480347
val Loss: 0.6925 Acc: 0.7838
Epoch time taken: 76.39903426170349

Epoch 18/24
----------
train Loss: 0.6325 Acc: 0.7927
Epoch time taken: 73.29569625854492
val Loss: 0.6652 Acc: 0.7857
Epoch time taken: 76.20719122886658
```

```
Epoch 19/24
----------
train Loss: 0.6076 Acc: 0.7959
Epoch time taken: 73.5692126750946
val Loss: 0.6353 Acc: 0.7905
Epoch time taken: 76.37571692466736

Epoch 20/24
----------
train Loss: 0.5792 Acc: 0.8055
Epoch time taken: 77.1331775188446
val Loss: 0.7029 Acc: 0.7657
Epoch time taken: 79.99154663085938

Epoch 21/24
----------
train Loss: 0.5560 Acc: 0.8142
Epoch time taken: 77.18012261390686
val Loss: 0.6715 Acc: 0.8000
Epoch time taken: 80.11594772338867

Epoch 22/24
----------
train Loss: 0.5407 Acc: 0.8155
Epoch time taken: 73.12362718582153
val Loss: 0.6286 Acc: 0.8010
Epoch time taken: 75.9416389465332

Epoch 23/24
----------
train Loss: 0.5085 Acc: 0.8305
Epoch time taken: 76.15018844604492
val Loss: 0.6683 Acc: 0.7876
Epoch time taken: 78.97465777397156

Epoch 24/24
----------
train Loss: 0.4794 Acc: 0.8433
Epoch time taken: 76.16518664360046
val Loss: 0.5891 Acc: 0.8200
Epoch time taken: 78.99133062362671

Training complete in 32m 40s
Best val Acc: 0.8200
```

[30]:
```
train_accs=to_float_list(train_accs)
val_accs=to_float_list(val_accs)
train_losses=to_float_list(train_losses)
val_losses=to_float_list(val_losses)

plot_curves(train_accs, val_accs, train_losses, val_losses, "Ex3 Custom ResNet␣
↪")
```





[31]:
```
show_predictions(model=resnet,
                 dataloader=test_loader,
                 class_names=train_dataset.classes,
                 device=device,
                 num_images=9)
```